Desarrollo e implementación con tecnologías libres de un dispositivo portátil para evaluar el confort térmico, lumínico y acústico en interiores

José Ramón Hernández Aguilar

2024-12-25

Tabla de contenidos

Portada **3**

1 Introducción al confort en espacios espacios interiores y el desarrollo del DTHIS-C **4**
1.1 Importancia del confort en espacios interiores 4
1.2 Modelos de confort adaptativos 4
1.3 DTHIS-C 4

2 Confort y calidad del aire en espacios interiores **5**
2.1 Confort térmico 5
2.2 Variables físicas del confort térmico 5
2.3 Confort acústico 6
2.4 Variables físicas del confort acústico 6
2.5 Confort lumínico 7
2.6 Variables físicas del confort lumínico 7
2.7 Calidad del aire 8
2.8 Variables físicas de la calidad del aire 8

3 Diseño y construcción del DTHIS-C **9**
3.1 Especificaciones del DTHIS-C 9
3.2 Diseño del DTHIS-C 10
3.2.1 Arduino UNO WiFi Rev2 10
3.2.2 Raspberry Pi 4 Model B 21
3.3 Lista de materiales 30

4 Calibración y caracterización del DTHIS-C **33**
4.1 Presentación de los resultados 33
4.2 Análisis e interpretación de resultados 33
4.3 Visualización de datos 33

5 Discusión y conclusiones **35**
5.1 Trabajo a futuro 35

Bibliografía **36**

Anexos **38**

Portada

1 Introducción al confort en espacios espacios interiores y el desarrollo del DTHIS-C

En este capítulo se examina la importancia del confort en espacios interiores conforme al ANSI/ASHRAE Standard 55, que establece las condiciones térmicas necesarias para el bienestar de los ocupantes. Se incluyen modelos adaptativos de confort, que explican cómo las personas ajustan sus expectativas de acuerdo con el clima y contexto, aportando claves para el diseño de entornos confortables y eficientes.

Se presenta también el DTHIS-C (Dispositivo de Temperatura, Humedad, Iluminación y Sonido de Campaña), un equipo portátil que mide variables ambientales como:

- Temperatura del aire a diferentes alturas
- Temperatura radiante
- Humedad relativa
- Velocidad del viento
- Concentración de CO2
- Luminancia
- Deslumbramiento
- Niveles de sonido

Destacando la importancia de su elaboración para el desarrollo de espacios interiores basados en criterios de confort térmico, acústico y lumínico.

1.1 Importancia del confort en espacios interiores

1.2 Modelos de confort adaptativos

1.3 DTHIS-C

2 Confort y calidad del aire en espacios interiores

En este capítulo se aborda el contexto del confort en espacios cerrados, incluyendo los aspectos térmico, acústico, lumínico y la calidad del aire. Se define el concepto de confort térmico, destacando las variables físicas que lo determinan. Además, se identifican y describen los principales fenómenos asociados con el disconfort térmico, acústico y lumínico. Finalmente, se presentan los aspectos fundamentales relacionados con el confort acústico, lumínico y la calidad del aire.

2.1 Confort térmico

El confort térmico se define como una condición mental que refleja la satisfacción con el entorno térmico, de acuerdo con el estándar ASHRAE-55 (2021).

Según Djongyang, Tchinda, y Njomo (2010), el confort térmico se alcanza cuando la temperatura corporal se mantiene dentro de rangos estrechos, la humedad en la piel es baja y el esfuerzo fisiológico para regular la temperatura es mínimo. Además, el confort no solo depende de factores ambientales, sino también de diferentes aspectos, como el tipo de vestimenta, el tipo de actividad física, cambiar de postura o ubicación, abrir una ventana, etc.

2.2 Variables físicas del confort térmico

Macpherson (1962) identificó seis factores determinantes que afectan la sensación térmica en los espacios cerrados. Entre ellos, cuatro son variables físicas: temperatura del aire, velocidad del aire, humedad relativa y temperatura media radiante; y dos son variables personales: el aislamiento térmico de la ropa y el nivel de actividad física, también conocido como tasa metabólica. Estos factores constituyen la base para el análisis y predicción del confort térmico.

Acorde a ASHRAE-55 (2021), dichas variables se definen de la siguiente manera:

1. **Temperatura del aire**: La temperatura medida en un punto específico del aire.
2. **Velocidad del aire**: La magnitud del movimiento del aire en un punto determinado, sin considerar su dirección.

3. **Humedad**: Representa el contenido de humedad en el aire y se expresa mediante varias variables termodinámicas, como presión de vapor, temperatura de punto de rocío, temperatura de bulbo húmedo, relación de humedad y humedad relativa. Su promedio espacial y temporal se calcula de manera similar al de la temperatura del aire.
4. **Temperatura media radiante (t_r)**: La temperatura de un recinto negro y uniforme que intercambia con el ocupante la misma cantidad de calor por radiación que el entorno real. Este valor único aplica a todo el cuerpo y considera tanto la temperatura radiante media de onda larga (t_{rlw}) como la de onda corta (t_{rsw}).
5. **Aislamiento térmico de la ropa (I_{cl})**: La resistencia al intercambio de calor sensible proporcionada por un conjunto de ropa, expresada en unidades de "clo". *Nota informativa*: Esta definición incluye la transferencia de calor de todo el cuerpo, incluyendo las áreas no cubiertas, como la cabeza y las manos.
6. **Tasa metabólica (met)**: La tasa a la que se transforma la energía química en calor y trabajo mecánico debido a las actividades metabólicas de un individuo. Se mide por unidad de área de superficie de la piel y se expresa en "met", equivalente a 58.2 W/m2, que corresponde a la energía generada por unidad de superficie de piel de una persona promedio en reposo.

2.3 Confort acústico

El confort acústico se define como "un estado de satisfacción con las condiciones acústicas", representa el bienestar y la percepción positiva de un entorno en términos sonoros. Sin embargo, este confort es altamente subjetivo, ya que fuentes de ruido con características físicas similares pueden percibirse de manera distinta dependiendo de la persona. Antoniadou y Papadopoulos (2017) Azar et al. (2020)

2.4 Variables físicas del confort acústico

Azar et al. (2020) nombra dos tipos principales de ruido en los edificios:

- **Ruido estructural (por impacto)**: Generado por impactos físicos o vibraciones sobre un elemento de la construcción.
- **Ruido aéreo**: Transmitido a través del aire.

El **nivel de presión sonora**, definido como la vibración del aire que constituye el sonido Berglund (1999), es uno de los principales factores acústicos que influyen en el confort. Un nivel neutral de presión sonora se encuentra en un rango de 30 a 40 dB Antoniadou y Papadopoulos (2017). Para predecir el confort frente al ruido de impacto, se utiliza típicamente el nivel

máximo de presión sonora (L_{max}), mientras que para el ruido aéreo se emplea el nivel de presión sonora equivalente durante un periodo de tiempo determinado (L_{eq}). Azar et al. (2020)

Asimismo Berglund (1999), enumera ciertos factores acústicos que impactan el confort acústico:

1. **Frecuencia del ruido**: Su unidad de medida son los Hertz (Hz) y se refiere al número de vibraciones por segundo del aire en el cual el sonido es propagado. El rango de frecuencia audible normalmente se encuentra en un rango de 20-20,000 Hz.
2. **Fuente del ruido**: Se mencionan diferentes fuentes, las cuales pueden ser industrial, de transporte, construcción y doméstico.
3. **Duración del ruido**
4. **Variación en el tiempo**: Existen variaciones de la presión sonora que suelen causar mayor disconfort auditivo. Por ejemplo, los ruidos que varían periódicamente para crear una sensación de pulsación pueden ser más molestos que el ruido continuo, las variaciones de aproximadamente 4 por segundo podrían ser más perturbadoras.

2.5 Confort lumínico

El confort lumínico se refiere a la percepción de bienestar visual en un espacio, permitiendo que las personas realicen sus actividades cotidianas de manera efectiva y cómoda bajo condiciones de iluminación adecuadas. Vásquez et al. (2019)

2.6 Variables físicas del confort lumínico

Según se describe en la norma ISO EN 12464-1 Standard (2011), la evaluación del confort lumínico puede lograrse mediante la determinación de: la luminancia y la iluminancia.

1. **Luminancia**: Su unidad de medida es la candela por metro cuadrado (cd/m^2) y se define como la densidad de la intensidad luminosa en una dirección específica, por unidad de área proyectada en un punto determinado de una superficie real o imaginaria. CIE (2020b)

   Matemáticamente, se expresa como:

   $L_v = \frac{I_v}{A \cdot \cos(\alpha)}$

   donde (L_v) es la luminancia, (I_v) representa la intensidad luminosa en la dirección especificada, (A) es el área considerada, y (α) es el ángulo entre la normal a la superficie en el punto especificado y la dirección considerada.

La luminancia cuantifica cuán brillante aparece una superficie cuando se observa desde una dirección específica, teniendo en cuenta tanto la intensidad de la luz emitida o reflejada como la geometría de la superficie.

2. **Iluminancia**: Su unidad de medida es el lux (lx) y se define como la densidad del flujo luminoso incidente por unidad de área en un punto de una superficie real o imaginaria. CIE (2020a)

   Matemáticamente, se expresa como:

   $E_v = \frac{\Phi_v}{A}$

   donde (E_v) es la iluminancia, (Φ_v) representa el flujo luminoso incidente, y (A) es el área sobre la cual incide el flujo luminoso.

   La iluminancia cuantifica la cantidad de luz que llega a una superficie específica, considerando tanto la intensidad de la luz como el área sobre la cual se distribuye.

2.7 Calidad del aire

De acuerdo con el estándar ASHRAE-62 (2001), se considera que el aire interior tiene una calidad aceptable cuando "no contiene contaminantes conocidos en concentraciones nocivas según lo establecido por las autoridades competentes y la mayoría sustancial de las personas expuestas (80% o más) no expresa insatisfacción".

2.8 Variables físicas de la calidad del aire

La calidad del aire interior se evalúa monitoreando la **concentración de CO2**, comparándola con los niveles exteriores (300-500 ppm). ASHRAE-62 (2001) recomienda mantener concentraciones interiores entre 1000 y 1200 ppm para garantizar un ambiente adecuado y regular los sistemas de ventilación. Sin embargo, concentraciones superiores a 5000 ppm son peligrosas, ya que indican una ausencia total de aire fresco.

3 Diseño y construcción del DTHIS-C

En este capítulo se presenta una descripción detallada del dispositivo DTHIS-C, incluyendo sus componentes y especificaciones técnicas. Además, se proporcionan los diagramas de conexión y los códigos necesarios para la configuración de cada sensor, facilitando tanto la reproducción completa del DTHIS-C como la implementación individual de sus sensores.

3.1 Especificaciones del DTHIS-C

En la tabla Tabla 3.1 se muestran los sensores y componentes electrónicos que conforman al DTHIS-C.

Tabla 3.1: Sensores y componentes electrónicos que consitutyen al DTHIS-C.

Sensor	**Variable**	**Comunicación**	**Microcontrolador**	**Acondicionador de señal**
Termopar tipo T	Temperatura ambiente	ICSP	Arduino UNO WiFi Rev2	PWFusion MAX31856 SEN-30007
TPF1/E-20 PT1000	Temperatura radiante	ICSP	Arduino UNO WiFi Rev2	Adafruit PT1000 RTD-MAX31865
Wind Sensor Rev P6	Velocidad del viento	ADC	Arduino UNO WiFi Rev2	
SCD30 Sensirion	CO2 y humedad relativa	I2C	Raspberry Pi 4 Model B	
5MP OV5647 Wide Angle Fisheye Camera	Luminancia	CSI	Raspberry Pi 4 Model B	
Micrófono ambiental USB	Sonido	USB	Raspberry Pi 4 Model B	

3.2 Diseño del DTHIS-C

La arquitectura del DTHIS-C se ha fundamentado en el uso combinado de la Raspberry Pi 4 y el Arduino Uno WiFi Rev2, microcontroladores seleccionados por las siguientes características y ventajas:

1. **Versatilidad y capacidad de procesamiento:**
   La Raspberry Pi 4, programada mediante Python, proporciona una plataforma robusta para el procesamiento avanzado de datos y la ejecución de algoritmos complejos. Su flexibilidad, junto con la facilidad de integración de diversas funcionalidades, permite adaptar el sistema a requerimientos específicos y optimizar el rendimiento según las demandas del entorno.

2. **Conectividad y eficiencia en la adquisición de datos:**
   Ambos dispositivos cuentan con conectividad a Internet, lo que posibilita la transmisión en tiempo real de datos hacia una base de datos remota alojada en el servidor ThingsBoard. Además, el Arduino Uno WiFi Rev2 destaca por su eficiencia en la gestión de operaciones de entrada/salida, garantizando una respuesta rápida y precisa en la adquisición y control de los sensores.

3. **Compatibilidad y ecosistema abierto:**
   La adopción de tecnologías abiertas facilita la integración y reproducibilidad del sistema. La amplia comunidad de desarrolladores que respalda tanto la Raspberry Pi como el Arduino asegura soporte continuo, actualizaciones regulares y una extensa variedad de recursos y librerías, lo que se traduce en mayor flexibilidad para adaptar el sistema a diversas necesidades y entornos.

4. **Accesibilidad:**
   Frente a equipos especializados, la utilización de estos microcontroladores constituye una solución rentable sin comprometer el rendimiento ni la fiabilidad del sistema. Esta ventaja económica facilita la replicación y el escalado del proyecto en distintos contextos.

A continuación, se detallan las especificaciones técnicas y el código implementado para cada sensor.

3.2.1 Arduino UNO WiFi Rev2

El Arduino UNO WiFi Rev2 es una placa de microcontrolador diseñada para proyectos de Internet de las Cosas (IoT), integrando conectividad WiFi y Bluetooth mediante el módulo u-blox NINA-W102 y un sensor IMU de 6 ejes (LSM6DS3TR) para detección de movimientos. Equipada con el microcontrolador ATmega4809, ofrece una óptima capacidad de memoria, y mantiene compatibilidad con el formato estándar UNO, facilitando su uso con shields existentes. Su diseño eficiente y versátil lo hace ideal para la adquisición de datos de sensores y la comunicación inalámbrica en tiempo real, como se requiere en el DTHIS-C. Arduino (2023)

En la tabla Tabla 3.2 se muestran sus especificaciones.

Tabla 3.2: Especificaciones del Arduino UNO WiFi Rev2.

Parámetro	**Detalle**
Microcontrolador	ATmega4809
Memoria flash	48 KB
SRAM	6,144 Bytes
EEPROM	256 Bytes
Conectividad Wi-Fi	Sí, u-blox NINA-W102 (2.4 GHz)
Bluetooth	Sí, Bluetooth Low Energy
Sensor IMU	Sí, LSM6DS3TR (6 ejes)
Pines digitales I/O	14 (5 PWM)
Pines analógicos	6
Voltaje de operación	5V
Consumo máximo total	200 mA
Dimensiones	68.6 mm x 53.4 mm

El siguiente código constituye el programa principal del Arduino UNO WiFi Rev2 en el DTHIS-C, encargado de gestionar la conexión WiFi, la comunicación con ThingsBoard mediante el protocolo MQTT, y el envío de datos provenientes de sensores como termopares, temperatura radiante y velocidad del viento:

```
#include <WiFiNINA.h>
#include <PubSubClient.h>

// Definiciones para la red WiFi y ThingsBoard
#define WIFI_AP "IER"
#define WIFI_PASSWORD "acadier2014"
#define TOKEN "bXQq1eepFYJV3nldvLTV"
#define THINGSBOARD_SERVER "tb.ier.unam.mx"
#define MQTT_PORT 1883

// Inicialización de las librerías y objetos
WiFiClient espClient;
PubSubClient mqttClient(espClient);

/**
 * Configuración inicial del dispositivo.
 * Configura la conexión WiFi y el cliente MQTT para ThingsBoard.
 */
void setup() {
```

```
  WiFi.begin(WIFI_AP, WIFI_PASSWORD);  // Conectar a la red WiFi
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);  // Espera hasta conectarse
  }

  mqttClient.setServer(THINGSBOARD_SERVER, MQTT_PORT);
  connectToThingsBoard();  // Conectar a ThingsBoard
}

/**
 * Ciclo principal del programa.
 * Mantiene las conexiones y realiza las tareas periódicas.
 */
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    reconnectWiFi();  // Reconecta WiFi si es necesario
  }

  if (!mqttClient.connected()) {
    reconnectThingsBoard();  // Reconecta MQTT si es necesario
  }

  // Obtener las mediciones de los termopares (temperaturas)
  float* temperatures = ThermocoupleTemp();
  sendTC(temperatures);    // Enviar temperaturas a ThingsBoard

  // Obtener la temperatura del TPF1
  float radiant = TPF1Temp();
  sendRT(radiant);  // Enviar temperatura radiante a ThingsBoard

  // Obtener la velocidad promedio del viento
  float* windspeed = WindSensor();
  sendWS(windspeed);      // Enviar velocidad del viento a ThingsBoard

  mqttClient.loop();
  delay(2000);  // Espera 2 segundos antes de la siguiente iteración
}

/**
 * Conecta a la red WiFi.
 */
void connectWiFi() {
```

```
  WiFi.begin(WIFI_AP, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);  // Espera hasta conectarse
  }
}

/**
 * Reconecta a la red WiFi si es necesario.
 */
void reconnectWiFi() {
  while (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(WIFI_AP, WIFI_PASSWORD);
    delay(500);
  }
}

/**
 * Reconecta a ThingsBoard si es necesario.
 */
void reconnectThingsBoard() {
  while (!mqttClient.connected()) {
    connectToThingsBoard();
    delay(5000);  // Espera antes de reintentar
  }
}

/**
 * Conecta a ThingsBoard usando el token de autenticación.
 */
bool connectToThingsBoard() {
  return mqttClient.connect("ArduinoClient", TOKEN, NULL);
}

/**
 * Envía los datos de temperatura.
 */
void sendTC(float* temperatures) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"T1\":" + String(temperatures[0]) +
                   ", \"T2\":" + String(temperatures[1]) +
                   ", \"T3\":" + String(temperatures[2]) +
                   ", \"T4\":" + String(temperatures[3]) + "}";
```

```
  mqttClient.publish(topic.c_str(), payload.c_str());
}

/**
 * Envía los datos de temperatura radiante.
 */
void sendRT(float radiant) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"TR\":" + String(radiant) + "}";
  mqttClient.publish(topic.c_str(), payload.c_str());
}

/**
 * Envía los datos de velocidad del viento.
 */
void sendWS(float* windspeed) {
  String topic = "v1/devices/me/telemetry";
  String payload = "{\"WS\":" + String(windspeed[0]) +
                   ", \"TEMP\":" + String(windspeed[1]) + "}";
  mqttClient.publish(topic.c_str(), payload.c_str());
}
```

3.2.1.1 Termopar Tipo T

En la construcción del DTHIS se emplea un termopar tipo T, un sensor de temperatura compuesto por la unión de cobre y cobre-níquel. Su uso es ideal para entornos con humedad.

A continuación, se presentan sus especificaciones según OMEGA (s. f.):

Rangos de Temperatura
- **Grado de termopar:** -200 a 350 °C (-328 a 662 °F).
- **Grado de extensión:** -60 a 100 °C (-76 a 212 °F).

Límites de Error
- **Estándar:**
- ±1.0 °C o ±0.75% por encima de 0 °C.
- ±1.0 °C o ±1.5% por debajo de 0 °C.
- **Especial:**
- ±0.5 °C o ±0.4%.

3.2.1.2 PWFusion MAX31856 SEN-30007

El PWFusion MAX31856 SEN-30007 es un shield de termopares de cuatro canales diseñado para integrarse con la plataforma Arduino. Este dispositivo se conecta directamente a la placa de desarrollo. El módulo acondiciona la señal proveniente de termopares, realizando el procesamiento necesario para adaptar y amplificar la señal eléctrica, la cual se caracteriza por su baja amplitud.

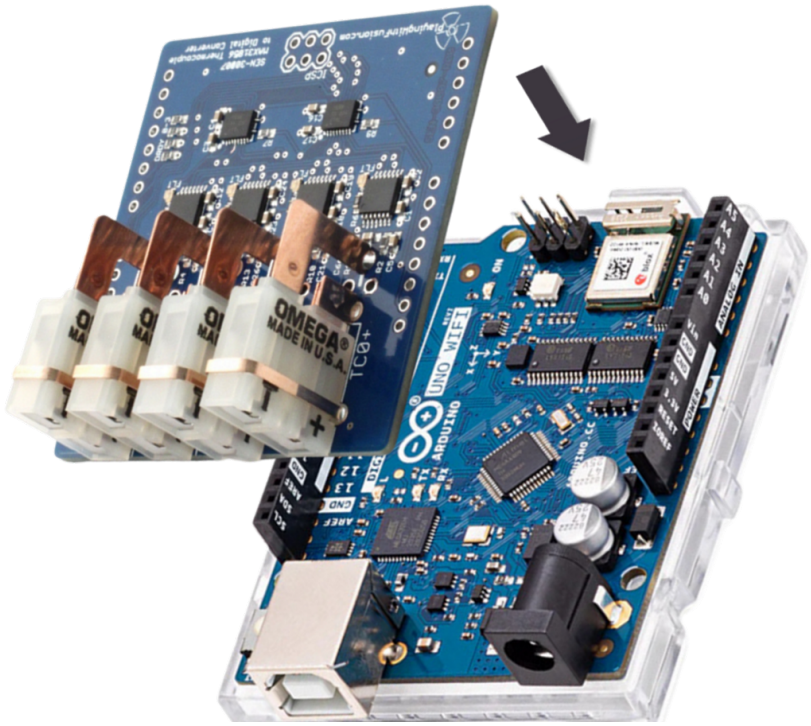

Figura 3.1: Diagrama de conexión para el shield MAX31856 SEN-30007

Acorde a PWFusion (s. f.) sus específicaciones son las siguientes:

- Conversión de señal analógica a digital con resolución de 19 bits.
- Interfaz ICSP de 4 hilos.
- Rango de voltaje de alimentación: 3.3 V a 5.0 V.

Igualmente se proporciona el código de Arduino empleado:

```
#include <PWFusion_MAX31856.h>

// Definición de los pines chip select para cada termopar
uint8_t tcChipSelects[] = {10, 9, 8, 7};
```

```
#define NUM_THERMOCOUPLES   (sizeof(tcChipSelects) / sizeof(uint8_t))

// Coeficientes de calibración para cada termopar
const float m[NUM_THERMOCOUPLES] = {1.0450, 1.0602, 1.0814, 1.0554};  // Pendientes
const float b[NUM_THERMOCOUPLES] = {-2.4327, -2.8632, -4.4316, -3.9348};  // Intersecciones

// Se crea un objeto MAX31856 para cada termopar
MAX31856 thermocouples[NUM_THERMOCOUPLES];

float* ThermocoupleTemp() {
  static float tc_temps[NUM_THERMOCOUPLES];
  static bool initialized = false;

  // Inicializa los sensores solo la primera vez que se llama a la función
  if (!initialized) {
    delay(1000);  // Espera para la estabilización de los sensores
    for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
      thermocouples[i].begin(tcChipSelects[i]);
      thermocouples[i].config(T_TYPE, CUTOFF_60HZ, AVG_SEL_4SAMP, CMODE_OFF);
    }
    initialized = true;
  }

  // Inicia la medición en cada sensor
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    thermocouples[i].startOneShotMeasurement();
  }

  // Espera el tiempo necesario para que la conversión se complete
  delay(180);

  // Actualiza la muestra en cada sensor
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    thermocouples[i].sample();
  }

  // Obtiene la temperatura ambiente de cada sensor y aplica la calibración
  for (int i = 0; i < NUM_THERMOCOUPLES; i++) {
    if (thermocouples[i].getStatus() != 0) {
      // En caso de error, se asigna NaN (Not a Number)
      tc_temps[i] = NAN;
    } else {
```

```
      float raw_temperature = thermocouples[i].getTemperature();
      tc_temps[i] = m[i] * raw_temperature + b[i];  // Aplica la calibración
    }
  }

  return tc_temps;
}
```

3.2.1.3 Fuehler Systeme TPF1/E-20 PT1000

Se empleó el TPF1/E-20 PT1000, diseñado para medir la temperatura radiante en un rango de temperaturas de -30 °C a +75 °C. Su sonda de temperatura, basada en la tecnología PT1000, se puede montar como un péndulo libremente suspendido, lo que permite obtener mediciones precisas de la temperatura de sensación térmica en entornos donde la convección natural y la estratificación del aire pueden influir en los resultados. FuehlerSysteme (s. f.)

Especificaciones

- Tipo de circuito: Conexión de 2 hilos.
- Corriente de medición: Aproximadamente 1 mA.
- Conexión eléctrica: Extremos pelados con terminales.
- Cable: Cable de PVC (2×0,25 mm2, temperatura máxima +105 °C) con extremos de cable núcleo disponibles en diferentes longitudes.
- Resistencia de fuga: Mayor a 100 MΩ a +20 °C (500 V DC).
- Material del globo: Aluminio (negro).
- Dimensiones del globo: Diámetro de 70 mm.

3.2.1.4 Adafruit PT1000 RTD-MAX31865

El PT1000 RTD-MAX31865 es un convertidor de resistencia a digital.

Adafruit (s. f.) describe las siguientes características:

- Realiza la conversión de la resistencia de sensores RTD de platino a un valor digital de forma sencilla.
- Soporte con RTD de platino con valores de 100 Ω a 1 kΩ (a 0 °C), abarcando desde PT100 hasta PT1000.
- Compatible con configuraciones de 2, 3 y 4 hilos para la conexión de sensores.
- Incorpora una interfaz ICSP, facilitando su integración en sistemas basados en microcontroladores.

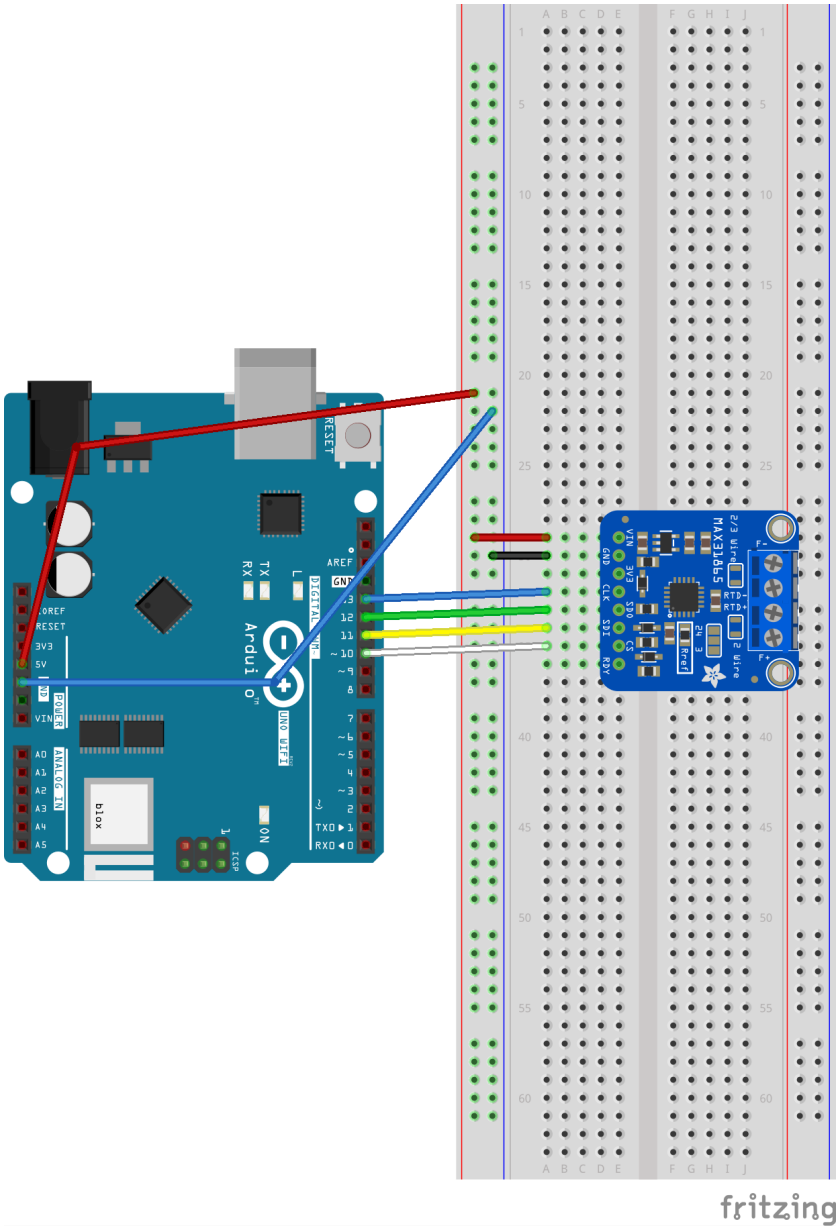

Figura 3.2: Diagrama de conexión para el PT1000 RTD-MAX31865

- Cuenta con un ADC de 15 bits que proporciona una resolución nominal de temperatura de 0.03125 °C (valor variable debido a la no linealidad del RTD) y una precisión total de hasta 0.5 °C (0.05% de la escala completa) en todas las condiciones operativas.
- Tiempo máximo de conversión es de 21 ms.

A continuación se muestra el código de Arduino:

```
#include <Adafruit_MAX31865.h>

// Usamos software SPI: CS, DI, DO, CLK
Adafruit_MAX31865 thermo = Adafruit_MAX31865(10, 11, 12, 13);
// Para hardware SPI solo se pasa el pin CS
// Adafruit_MAX31865 thermo = Adafruit_MAX31865(10);

#define RREF      4300.0  // Resistor de referencia (usa 430.0 para PT100 o 4300.0 para PT10
#define RNOMINAL  1000.0  // Resistencia nominal a 0°C (100.0 para PT100, 1000.0 para PT1000

// Función que realiza la medición y retorna la temperatura radiante en grados Celsius.
float TPF1Temp() {
  // Inicializa el sensor solo la primera vez que se llama a la función.
  static bool initialized = false;
  if (!initialized) {
    thermo.begin(MAX31865_2WIRE);  // Inicialización del MAX31865
    initialized = true;
  }

  // Lectura del valor RTD
  uint16_t rtd = thermo.readRTD();

  // Calcula la razón y la resistencia (opcional si se requiere para otros cálculos)
  float ratio = rtd / 32768.0;
  float resistance = RREF * ratio;

  // Calcula la temperatura utilizando el método de la librería
  float radiant_temp = thermo.temperature(RNOMINAL, RREF);

  return radiant_temp;
}
```

3.2.1.5 Modern Device Wind Sensor Rev. P6

El sensor de viento Rev. P6 es un anemómetro de hilo caliente. Incorpora un potenciómetro de alta precisión para facilitar la calibración (realizada en fábrica) y utilizar termistores de coeficiente de temperatura positivo, que aseguran mediciones más estables incluso ante variaciones de temperatura ambiente. Requiere una fuente de alimentación de 9 a 12 V (idealmente 12 V) para garantizar el adecuado calentamiento de los termistores y evitar la saturación en condiciones de viento intenso. Integra un sensor de temperatura ambiental con salida escalada a 3.3 V y una señal de viento ajustada a un máximo de 3.3 V mediante resistencias de alta precisión. ModernDevice (s. f.)

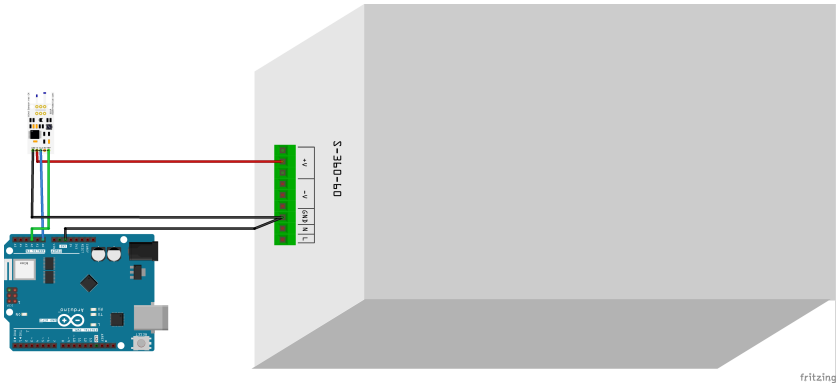

Figura 3.3: Diagrama de conexión para el Wind Sensor Rev. P6

Especificaciones

- Voltaje de alimentación: 10–12 V.
- Corriente: Aproximadamente 40 mA.
- Velocidades de viento medidas: 0–150 mph.
- Compensación de temperatura ambiente.

```
#include <math.h>

const int OutPin  = A0;   // Pin analógico para el sensor de viento ("OUT")
const int TempPin = A2;   // Pin analógico para el sensor de temperatura ("TMP")

// Función que lee velocidad de viento y temperatura, y retorna un array de dos floats:
// ws[0] -> velocidad de viento (m/s)
// ws[1] -> temperatura (°C)
float* WindSensor() {
  static float ws[2];

  // Lectura del sensor de viento
```

```
  int windADunits = analogRead(OutPin);
  float voltage = (float)windADunits * 5.0 / 1024.0;

  // Aplicar la corrección al voltaje
  float correctedVoltage = voltage - 1.1621;

  // Lectura del sensor de temperatura
  int tempRawAD = analogRead(TempPin);
  float tempC = ((((float)tempRawAD * 5.0) / 1024.0) - 0.400) / 0.0195;

  // Cálculo de la velocidad de viento en m/s usando la ecuación empírica:
  // WS = 26.3431 * (correctedVoltage)^(1.4273) * (tempC)^(-0.7631)
  float windSpeed = 26.3431 * pow(correctedVoltage, 1.4273) * pow(tempC, -0.7631);

  ws[0] = windSpeed;
  ws[1] = tempC;

  return ws;
}
```

3.2.2 Raspberry Pi 4 Model B

El Raspberry Pi 4 Model B es una computadora de placa única de alto rendimiento, diseñada para tareas de procesamiento avanzado y conectividad eficiente en un formato compacto. Equipada con un procesador de 64 bits de cuatro núcleos y opciones de RAM de hasta 8 GB, ofrece soporte para salida de video dual 4K y conectividad avanzada mediante Wi-Fi de doble banda, Bluetooth 5.0, Ethernet Gigabit y puertos USB 3.0, con capacidad para Power over Ethernet (PoE) mediante un accesorio adicional. Su diseño versátil y eficiente la hace ideal para la integración y gestión de datos en el sistema DTHIS-C. Raspberry (2019)

En la tabla Tabla 3.3 se muestran sus especificaciones.

Tabla 3.3: Especificaciones del Raspberry Pi 4 Model B.

Parámetro	**Detalle**
Procesador	Broadcom BCM2711, 4 núcleos Cortex-A72, 1.5 GHz
Memoria RAM	8 GB (LPDDR4-3200)
Conectividad Wi-Fi	Sí, dual-band 2.4 GHz y 5 GHz, 802.11ac
Bluetooth	Sí, Bluetooth 5.0
Puertos USB	2 USB 3.0, 2 USB 2.0
Salida de video	2 micro HDMI, soporte dual 4K 60 fps
Ethernet	Gigabit

Parámetro	Detalle
Almacenamiento	Tarjeta microSD
Voltaje de operación	5V
Consumo energético	Eficiente, sin ventilador (~3A recomendado)
Dimensiones	85.6 mm x 56.5 mm x 17 mm

El siguiente código constituye el programa principal del Raspberry Pi 4 Model B en el DTHIS-C, encargado de gestionar la conexión al servidor ThingsBoard mediante el protocolo MQTT y el envío de datos provenientes de sensores como el SCD30 (CO2, temperatura y humedad relativa). Escrito en Python, utiliza la librería `paho-mqtt` para la comunicación y está diseñado para operar de manera continua::

```
import paho.mqtt.client as mqtt
import json
import time
from thingsboard_credentials import thingsboard_data
from scd30 import scd30_measurements
from sound import sound_data

# Obtiene los datos de conexión para ThingsBoard
unique_id, token, thingsboard_host = thingsboard_data()

# Diccionario para almacenar las lecturas del sensor
sensor_data = {'CO2': 0, 'T': 0, 'HR': 0}
# Diccionario para datos de sonido
# micro_data = {'RMS': 0, 'dBmax': 0, 'dBmin': 0}

# Configura el cliente MQTT
client = mqtt.Client(unique_id, False)
client.username_pw_set(token, password=None)
client.connect(thingsboard_host, 1883, 60, '')
client.loop_start()

try:
    while True:
        # Obtiene las mediciones del sensor SCD30
        CO2, T, HR = scd30_measurements()

        # Actualiza el diccionario con las lecturas actuales
        sensor_data['CO2'] = CO2
        sensor_data['T'] = T
```

```
        sensor_data['HR'] = HR

        # Publica los datos del SCD30 en ThingsBoard
        client.publish('v1/devices/me/telemetry', json.dumps(sensor_data), 1)

        # Obtiene los datos de sonido
        # micro_data = sound_data()

        # Publica los datos de sonido en ThingsBoard
        # client.publish('v1/devices/me/telemetry', json.dumps(micro_data), 1)

        time.sleep(2)  # Ajusta el intervalo de tiempo entre lecturas

except KeyboardInterrupt:
    pass

# Detiene el loop del cliente MQTT y desconecta del servidor
client.loop_stop()
client.disconnect()
```

3.2.2.1 Sensirion SCD30

El SCD30 es un sensor de CO2 basado en tecnología de detección por infrarrojos. Incorpora un sensor de humedad y temperatura. Cuenta con un diseño de doble canal. Además, ofrece una precisión de ±(30 ppm + 3% de la lectura) en un rango de 400 a 10,000 ppm. Sensirion (s. f.)

Especificaciones

CO2

- Rango de medición: 400 – 10,000 ppm
- Precisión: ±30 ppm ±3% del valor medido
- Tiempo de respuesta (63%): 20 s
- Estabilidad térmica: 2.5 ppm/°C

Humedad

- Precisión típica: ±3 %RH
- Rango de operación: 0 – 95 %RH
- Tiempo de respuesta (63%): 8 s
- Certificación de calibración: Calibrado de fábrica
- Temperatura

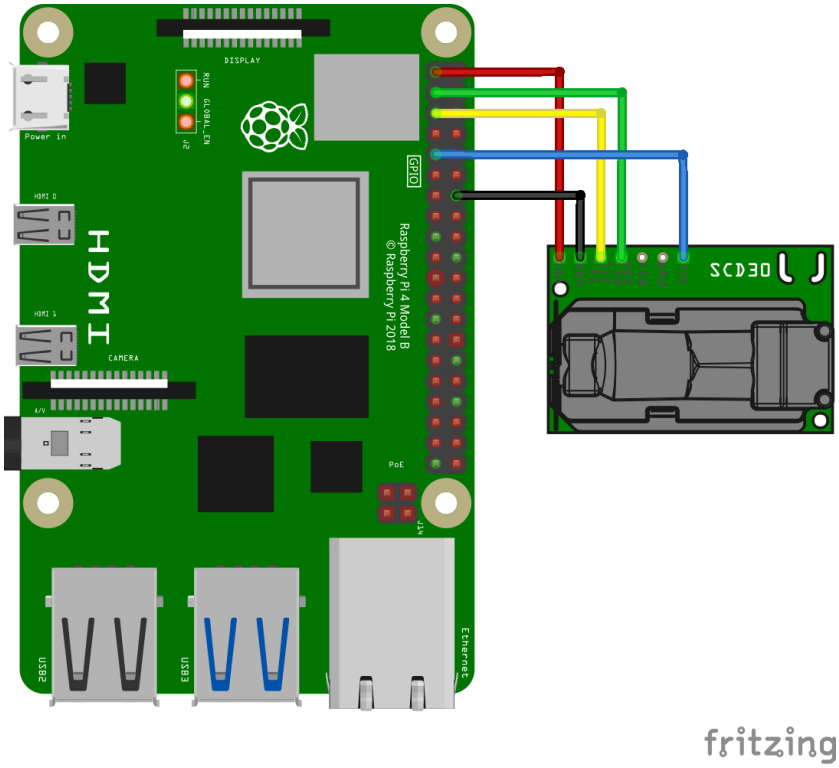

Figura 3.4: Diagrama de conexión para el SCD30

- Precisión típica: ±0.4 °C
- Tiempo de respuesta (63%): 10 s

Características generales

- Voltaje de alimentación: 3.3 – 5.5 V
- Corriente promedio: 19 mA
- Corriente máxima: 75 mA
- Rango de temperatura de operación: 0 – 50 °C
- Interfaces: I²C, ModBus, PWM
- Dimensiones: 35 × 23 × 7 mm

A continuación se muestra el código de Python empleado para us incorpación al DTHIS-C:

```
import time
import board
import busio
from scd30_i2c import SCD30

def scd30_measurements():
    """
    Función para obtener las lecturas de CO2, Temperatura y Humedad Relativa del sensor SCD30

    Retorna:
        tuple: (CO2, T, HR) donde
            CO2: concentración de CO2 en ppm
            T: temperatura en grados Celsius
            HR: humedad relativa en porcentaje
    """
    # Inicializa el sensor SCD30
    scd30 = SCD30()
    # Espera para asegurarse de que el sensor esté listo
    time.sleep(2)

    # Verifica si los datos están listos para ser leídos
    if scd30.get_data_ready():
        # Lee las mediciones del sensor
        CO2, T, HR = scd30.read_measurement()
        # Redondea cada valor a dos decimales
        CO2 = round(CO2, 2)
        T = round(T, 2)
        HR = round(HR, 2)
        return CO2, T, HR
```

3.2.2.2 Arducam 5MP OV5647 Ultra Wide Angle

La cámara Arducam Ultra Wide Angle cuenta con un objetivo ojo de pez M12 que ofrece un campo de visión horizontal de 220°. Arducam (s. f.)

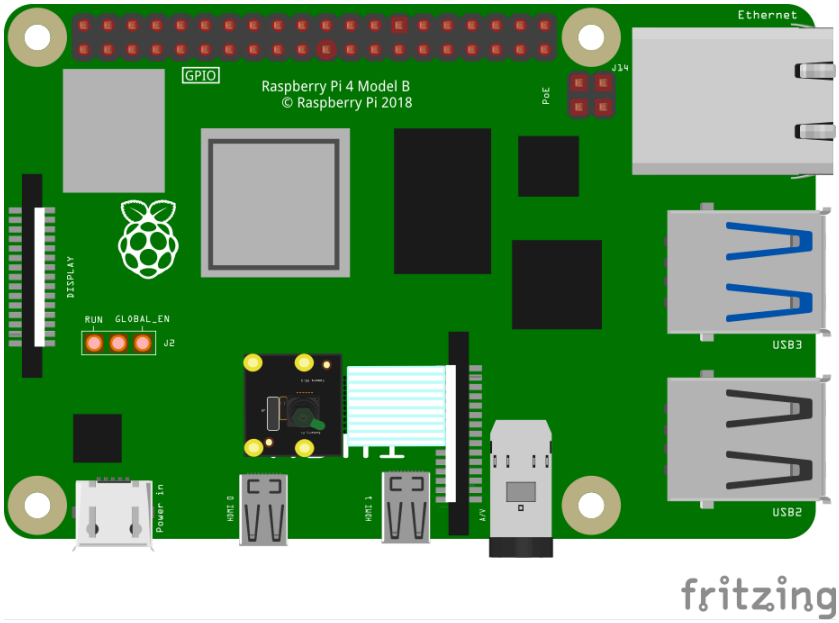

Figura 3.5: Diagrama de conexión para la cámara OV5647

Especificaciones

- Sensor: Omnivision OV5647
- Tamaño del sensor: 1/4 (OV5647)
- Resolución: 5 MP (2592×1944)
- Video:
  - 1080p a 30 fps
  - 720p a 60 fps
  - 480p a 90 fps
- Sensibilidad IR: Filtro IR-cut integrado (solo luz visible)
- Campo de visión horizontal (HFOV): 220°
- Distancia focal efectiva: 0.76 mm
- Distancia focal equivalente en 35 mm: 8.2 mm
- Tipo de enfoque: Manual
- Montura del objetivo: M12
- Dimensiones: 36 mm × 36 mm
- Corriente pico: 300 mA

A continuación, se muestra el código escrito en Bash/Shell que accede a la cámara y, posteriormente, genera los mapas de luminancia:

```
#!/bin/bash

DATE=$(date +"%Y-%m-%d_%H%M")
IMG_DIR="/home/hdeza/Luminance/images"
HDR_IMG="/home/hdeza/Luminance/HDR_images"
MAPS_DIR="/home/hdeza/Luminance/maps"
OUTPUT_DIR="/home/hdeza/Luminance"

#Loop through shutter speeds and capture images
SHUTTERS=(100 500 1000 5000 10000 50000 100000 500000 1000000 2000000)

for i in "${!SHUTTERS[@]}"; do
  rpicam-still --raw -n --gain 1 -t 500 --shutter "${SHUTTERS[$i]}" -o "$IMG_DIR/$(printf "%(
done

#Run raw2hdr with the path to the images and save as image.hdr
raw2hdr -a -e -g -f -h -w -o "$HDR_IMG/image_$DATE.hdr" "$IMG_DIR"/*dng

#Get info about the final HDR
getinfo "$HDR_IMG/image_$DATE.hdr"

#Nullify exposure value
ra_xyze -r -o "$HDR_IMG/image_$DATE.hdr" > "$OUTPUT_DIR/image_nullEV_$DATE.hdr"

#Resize image
pfilt -1 -x 1000 -y 1000 "$OUTPUT_DIR/image_nullEV_$DATE.hdr" > "$OUTPUT_DIR/image_resize_$DA

#Make photometric adjustment
pcomb -s 1.8 "$OUTPUT_DIR/image_resize_$DATE.hdr" > "$OUTPUT_DIR/image_photometric_$DATE.hdr"

#Change view angle for fisheye
getinfo -a "VIEW= -vta -vv 160 -vh 160" < "$OUTPUT_DIR/image_photometric_$DATE.hdr" > "$IMG_I

#Print illuminance value
echo "Total illuminance is: "
evalglare -V "$IMG_DIR/image_final_$DATE.hdr"
evalglare -V "$IMG_DIR/image_final_$DATE.hdr" > "$OUTPUT_DIR/illuminance_$DATE.txt"

#Move final HDR image to HDR_images
mv "$IMG_DIR/image_final_$DATE.hdr" "$HDR_IMG/image_final_$DATE.hdr"
```

```
#Crop to a 160
pcomb -e 'Cx:xmax/2;Cy:ymax/2;R:444.44;sq(x):x*x' -e 'inC=sq(R)-sq(x-Cx)-sq(y-Cy)' -e 'ro=if

#Make the luminance map with configurations and save it
falsecolor -s 5000 -d 1 -i "$HDR_IMG/image_crop_$DATE.hdr" --log 3 > "$MAPS_DIR/image_map_$DA
```

3.2.2.3 Micrófono ambiental USB

El micrófono incorpora un filtro que atenúa eficazmente los ruidos producidos por el viento o la respiración. Su patrón de captación direccional optimiza la focalización de la fuente sonora. Steren (s. f.)

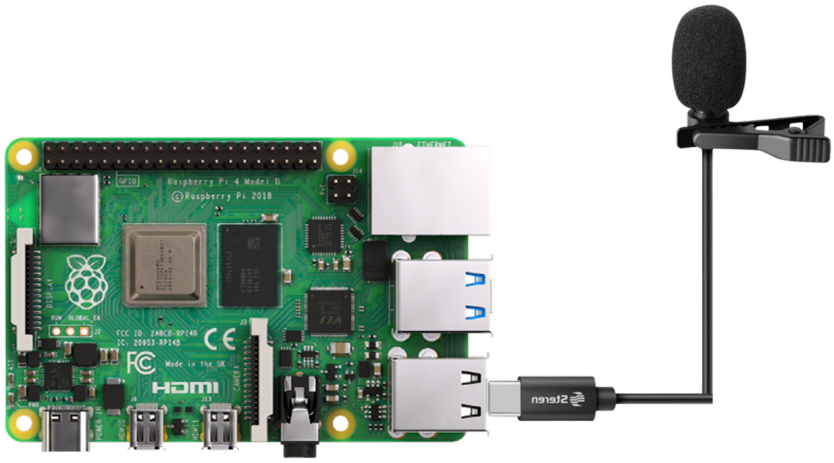

Figura 3.6: Diagrama de conexión para el micrófono

Especificaciones

- Respuesta en frecuencia: 50 a 10,000 Hz
- Impedancia 2,200 Ω
- Sensibilidad: -32 dB ±3 dB

A continuación, se presentan los códigos utilizados para la captura y análisis de sonido. Estos scripts permiten grabar el audio, extraer parámetros clave de la señal y convertirlos a valores en decibeles (dB).

El siguiente script graba un archivo de audio en formato WAV durante 15 segundos a una frecuencia de muestreo de 44.1 kHz:

```bash
#!/bin/bash

grabar="/home/hdeza/Sonido/audio.wav"

arecord -D plughw:3,0 -f cd -t wav -d 15 -r 44100 $grabar
```

Posteriormente este script analiza el archivo de audio y extrae la amplitud máxima de la señal, almacenándola en un archivo de texto:

```bash
#!/bin/bash

maximo="/home/hdeza/Sonido/dBmax.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "Maximum amplitude:"
```

Similar al anterior, este script obtiene la amplitud mínima del audio y la guarda en un archivo de texto:

```bash
#!/bin/bash

minimo="/home/hdeza/Sonido/dBmin.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "Minimum amplitude:"
```

Este código calcula la amplitud RMS (Root Mean Square) del audio, un indicador de la potencia promedio de la señal sonora:

```bash
#!/bin/bash

media="/home/hdeza/Sonido/rms.txt"

/usr/bin/sox /home/hdeza/Sonido/audio.wav -n stat 2>&1 | /usr/bin/grep "RMS     amplitude:"
```

El código en Python lee los valores extraídos previamente (RMS, amplitud máxima y mínima) y los convierte a decibeles utilizando la fórmula:

$$dB = 20 \log_{10}(\text{amplitud}) + 120$$

Los valores resultantes son almacenados en un diccionario para su posterior uso:

```python
import numpy as np

def sound_data():
    """
    Función para obtener los datos de sonido desde archivos y calcular sus valores en dB.

    Retorna:
        dict: Datos de sonido en dB (RMS_dB, dBmax_dB, dBmin_dB).
    """
    archivos = [
        '/home/dthisc/dthis-c/sound/rms.txt',
        '/home/dthisc/dthis-c/sound/dBmax.txt',
        '/home/dthisc/dthis-c/sound/dBmin.txt'
    ]

    rms_x = ""
    dBmax_x = ""
    dBmin_x = ""

    for archivo in archivos:
        with open(archivo, 'r', encoding='utf-8') as f:
            if archivo == '/home/dthisc/dthis-c/sound/rms.txt':
                rms_x = round(float(f.read().strip()), 4)
            elif archivo == '/home/dthisc/dthis-c/sound/dBmax.txt':
                dBmax_x = round(float(f.read().strip()), 4)
            elif archivo == '/home/dthisc/dthis-c/sound/dBmin.txt':
                dBmin_x = round(float(f.read().strip()), 4)

    rms_db = round(20 * np.log10(float(rms_x)) + 120, 4)
    dBmax_db = round(20 * np.log10(float(dBmax_x)) + 120, 4)
    dBmin_db = round(20 * np.log10(abs(float(dBmin_x))) + 120, 4)

    return {'RMS': rms_db, 'dBmax': dBmax_db, 'dBmin': dBmin_db}
```

3.3 Lista de materiales

La tabla Tabla 3.4 muestra la lista de materiales necesarios para la construcción del DTHIS-C, detallando la cantidad de cada componente, los costos y la fuente de venta.

Tabla 3.4: Lista de materiales para la construcción del DTHIS-C.

Designación	Componente	Cantidad	Costo unitario (MXN)	Costo total (MXN)	Fuente de compra
Adaptador USB	UGREEN Adaptador USB C Hembra a USB Macho 3	1	$199.00	$199.00	Amazon
Arduino	Arduino UNO WiFi Rev2	1	$1,268.10	$1,268.10	AG Electrónica
Cable para cámara Raspberry	OKY9053	1	$27.59	$27.59	AG Electrónica
Carcasa	Carcasa ABS para Raspberry Pi 4 con ventilador y disipadores de calor	1	$188.00	$188.00	330ohms
Fisheye	5MP OV5647 Wide Angle Fisheye Camera	1	$502.59	$502.59	AG Electrónica
Fuente de alimentación	DC 12 V 30 A Fuente de alimentación 360 W	1	$382.00	$382.00	Amazon
Fuente para Arduino	Aclorol 9V 1A DC Fuente de alimentación	2	$269.00	$538.00	Amazon
Fuente para Raspberry	Fuente oficial para Raspberry Pi 27W USB-C	1	$370.00	$370.00	330ohms
MAX31856	PWFusion SEN-30007 MAX31856	1	$1,493.10	$1,493.10	Playing With Fusion
MAX31865	Adafruit PT1000 RTD-MAX31865	1	$269.10	$269.10	Adafruit

Designación	Componente	Cantidad	Costo unitario (MXN)	Costo total (MXN)	Fuente de compra
Micro SD	Memoria Microsd Kingston Canvas Go Plus 64gb U3 V30 A2 170mb	1	$194.00	$194.00	Mercado Libre
Micrófono	Micrófono ambiental USB	1	$99.00	$99.00	Steren
Raspberry	Raspberry Pi 4 Model B 8 GB	1	$1,990.52	$1,990.52	AG Electrónica
SCD30	SCD30 Sensirion	1	$563.94	$563.94	DigiKey
Termopar	Termopar tipo T	30 m	$3,735.00	$3,735.00	Omega
TPF1	TPF1/E-20 PT1000	1	$1,576.60	$1,576.60	FuehlerSysteme
Tripié	XXZU 210 cm soporte de luz	1	$539.10	$539.10	Amazon
Wind Sensor	Wind Sensor Rev P6	1	$719.10	$719.10	Modern Device

Nota: Los precios no incluyen envío y pueden estar sujetos a cambios según el destino del producto.

4 Calibración y caracterización del DTHIS-C

En este capítulo se detalla el proceso de calibración y caracterización del dispositivo DTHIS-C. Se describen las metodologías experimentales empleadas para validar su precisión, así como las pruebas realizadas en diferentes escenarios. Además, se presentan y analizan los resultados obtenidos en términos de variables térmicas, lumínicas y acústicas, ofreciendo una evaluación integral de su desempeño. Finalmente, se explican las herramientas y procedimientos para la recuperación, visualización e interpretación de los datos recopilados.

4.1 Presentación de los resultados

4.2 Análisis e interpretación de resultados

4.3 Visualización de datos

```
import thesis_tools as tt

# Scatter plot desde un archivo Parquet
tt.scatter_plot('data/scd30.parquet', ['HR', 'T', 'CO2', 'HR_F', 'T_F', 'CO2_F'])
print('\n')

# Calcular errores con resampleo
tt.calculate_error('data/trad.parquet', 'temp', 'globo', resample_interval='1s')
print('\n')

# Calibrar windsensor
tt.wind_calibration('data/ws.parquet', ws='ws', v='voltage', t='temp', resample_interval='10s
print('\n')

# Calibrar termopares con intervalos de tiempo
time_intervals = [("15:44:01", "15:52:31", 10.),
                  ("16:02:06", "16:11:52", 20.),
                  ("16:20:44", "16:28:58", 30.),
```

```
                  ("16:39:23", "16:49:57", 40.),
                  ("16:58:24", "17:08:25", 50.)]
columns = ['TEMP1', 'TEMP2', 'TEMP3', 'TEMP4']
tt.thermo_calibration('data/temp.parquet', columns, time_intervals)
```

```
Unable to display output for mime type(s): text/html
```

```
ME: -0.1124
MAE: 0.1518
```

```
Correlación: 0.8637
WS = 26.3431 * voltage^(1.4273) * temp^(-0.7631)
```

```
TEMP1:
Tref = 1.0450 * TEMP1 + -2.4327

TEMP2:
Tref = 1.0602 * TEMP2 + -2.8632

TEMP3:
Tref = 1.0814 * TEMP3 + -4.4316

TEMP4:
Tref = 1.0554 * TEMP4 + -3.9348
```

5 Discusión y conclusiones

5.1 Trabajo a futuro

Bibliografía

Adafruit. s. f. «Adafruit PT1000 RTD Temperature Sensor Amplifier - MAX31865». https://www.adafruit.com/product/3648#description.

Antoniadou, Panagiota, y Agis M. Papadopoulos. 2017. «Occupants' thermal comfort: State of the art and the prospects of personalized assessment in office buildings». *Energy and Buildings* 153: 136-49. https://doi.org/10.1016/j.enbuild.2017.08.001.

Arducam. s. f. «Arducam 5MP OV5647 Ultra Wide Angle Fisheye Camera for Raspberry Pi». https://www.arducam.com/product/arducam-ultra-wide-angle-fisheye-5mp-ov5647-camera-for-raspberry-pi/.

Arduino. 2023. «Arduino UNO WiFi Rev2». https://store.arduino.cc/products/arduino-uno-wifi-rev2.

ASHRAE-55. 2021. *ANSI/ASHRAE Standard 55-2020: Thermal Environmental Conditions for Human Occupancy*. Atlanta, GA: American Society of Heating, Refrigerating; Air-Conditioning Engineers (ASHRAE).

ASHRAE-62. 2001. *ANSI/ASHRAE Standard 62-2001: Ventilation for acceptable indoor air quality*. Vol. 62. 2001. American Society of Heating, Refrigerating; Air-Conditioning Engineers.

Azar, Elie, William O'Brien, Salvatore Carlucci, Tianzhen Hong, Andrew Sonta, Joyce Kim, Maedot S. Andargie, et al. 2020. «Simulation-aided occupant-centric building design: A critical review of tools, methods, and applications». *Energy and Buildings* 224: 110292. https://doi.org/10.1016/j.enbuild.2020.110292.

Berglund, B. 1999. «Guidelines for community noise». *World Health Organization*.

CIE. 2020a. «Illuminance (17-21-060)». https://cie.co.at/eilvterm/17-21-060.

———. 2020b. «Luminance (17-21-050)». https://cie.co.at/eilvterm/17-21-050.

Djongyang, Noël, René Tchinda, y Donatien Njomo. 2010. «Thermal comfort: A review paper». *Renewable and Sustainable Energy Reviews* 14 (9): 2626-40. https://doi.org/10.1016/j.rser.2010.07.040.

FuehlerSysteme. s. f. «Radiation Pendulum Temperature Sensor». https://www.fuehlersysteme.de/Radiation-Pendulum-Temperature-Sensor_1#70bd/embedded/.

Macpherson, R. K. 1962. «The assessment of the thermal environment: A review». *British Journal of Industrial Medicine* 19 (3): 151-64. https://doi.org/10.1136/oem.19.3.151.

ModernDevice. s. f. «Wind Sensor Rev. P». https://moderndevice.com/products/wind-sensor-rev-p.

OMEGA. s. f. «¿Qué es un sensor termopar?» https://es.omega.com/prodinfo/termopares.html#.

PWFusion. s. f. «4-Channel T-Type Thermocouple MAX31856 SPI Digital Arduino Shield». https://www.playingwithfusion.com/productview.php?pdid=71&catid=1008.

Raspberry. 2019. «Raspberry Pi 4 Model B». https://www.raspberrypi.com/products/raspberry-pi-4-model-b/.

Sensirion. s. f. «SCD30». https://sensirion.com/products/catalog/SCD30.

Standard, British. 2011. «Light and lighting: Lighting of work places: Part 1: Indoor work places». *NS-EN 12464-1: 2011*.

Steren. s. f. «Micrófono USB C de solapa para celular». https://www.steren.com.mx/microfono-usb-c-de-solapa-para-celular.html.

Vásquez, Natalia Giraldo, Maíra Longhinotti Felippe, Fernando O. R. Pereira, y Ariane Kuhnen. 2019. «Luminous and visual preferences of young children in their classrooms: Curtain use, artificial lighting and window views». *Building and Environment* 152: 59-73. https://doi.org/10.1016/j.buildenv.2019.01.049.

Anexos

This is a book created from markdown and executable code.